Docs-as-Code Pipeline

Sample type

Docs-as-code tooling showcase — Describes the publishing workflow I designed: Git-managed Markdown, CI/CD automation, and CMS integration via custom tooling.

Transparency

The Python automation scripts described below were developed with the help of AI coding agents. I designed all the workflows, defined the requirements, and own the end-to-end operation of the pipeline.

Overview

I designed and implemented a complete docs-as-code publishing pipeline to manage 320+ articles across 6 API documentation products. The system treats documentation as source code — version-controlled, peer-reviewed, and automatically deployed.

```
┌──────────────┐     ┌──────────────┐     ┌────────────────┐     ┌──────────────┐
│  Write in    │──►│  Git commit  │──►│  CI/CD pipeline │──►│
Document360  │
│  Markdown    │     │  & push      │     │  runs deploy   │     │  (live
docs)  │
└──────────────┘     └──────────────┘     └────────────────┘     └──────────────┘
```

Source format

All documentation is authored in **Markdown** with **YAML frontmatter** that maps each file to its CMS article:

```yaml
---
title: "Create a booking overview"
article_id: "4ce5b020-9eda-461b-bbf7-72e1f90164ea"
```

```
category_id: "96811597-11e8-4390-9b2e-89a6f8eddf84"
version_id: "d735b0d0-7150-42f3-b7b1-5a0453235077"
---
```

The local folder structure mirrors the CMS category tree, making navigation intuitive for both authors and the CI/CD tooling.

Version control

The documentation repository is managed in **Git** (hosted on both GitHub and GitLab). All changes follow a branch-and-merge workflow:

1. Author creates a feature branch for the change.
2. Changes are committed with descriptive messages.
3. Pull/merge request is reviewed.
4. On merge to `main`, the CI/CD pipeline auto-deploys only the changed files.

CI/CD pipelines

Two pipelines were implemented — one for GitHub Actions and one for GitLab CI — ensuring the workflow works regardless of the hosting platform.

GitHub Actions

```yaml
name: Deploy Documentation
on:
  push:
    branches:
      - main
    paths:
      - '**/*.md'
      - 'doc360_article_map.json'

jobs:
  doc360-deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Code
        uses: actions/checkout@v3
        with:
          fetch-depth: 2

      - name: Set up Python
```

```yaml
        uses: actions/setup-python@v4
        with:
          python-version: '3.x'

      - name: Install Dependencies
        run: pip install requests

      - name: Run Deploy Sync
        env:
          DOC360_API_KEY_GET: ${{ secrets.DOC360_API_KEY_GET }}
          DOC360_API_KEY_POST: ${{ secrets.DOC360_API_KEY_POST }}
          DOC360_API_KEY_PUT: ${{ secrets.DOC360_API_KEY_PUT }}
          DOC360_API_KEY_DELETE: ${{ secrets.DOC360_API_KEY_DELETE }}
          DOC360_USER_ID: ${{ secrets.DOC360_USER_ID }}
        run: python3 doc360_manager.py ci-sync --commit-range HEAD~1
```

GitLab CI

```yaml
stages:
  - deploy

doc360-sync:
  stage: deploy
  image: python:3.11-slim
  rules:
    - if: '$CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH'
  variables:
    GIT_DEPTH: 5
  before_script:
    - apt-get update && apt-get install -y git
    - pip install -r requirements.txt
  script:
    - python doc360_manager.py ci-sync
        --commit-range "$CI_COMMIT_BEFORE_SHA..$CI_COMMIT_SHA"
```

Key design decisions:

- **Least-privilege API keys**: Separate secrets for GET, POST, PUT, and DELETE operations.
- **Change detection**: Only modified `.md` files are deployed, using `git diff` to identify changes.
- **Rate limiting**: The deploy script includes built-in rate limiting to stay within API quotas.

CMS integration tool

The core of the pipeline is a Python CLI tool (~900 lines) that provides bidirectional sync between the local Markdown files and the Document360 CMS.

Capabilities

Command	Description
`sync`	Pull remote content down to local Markdown files
`deploy`	Push local changes up (smart upsert with version forking)
`ci-sync`	CI/CD mode — deploy only files changed in the commit range
`list-versions`	List all project versions
`list-categories`	List the category tree
`list-articles`	List all articles in a version
`create-article`	Create a new article from a Markdown file
`publish-article`	Publish an article version

Smart upsert flow

When deploying an article, the tool follows this logic:

1. **Read** the local Markdown file and parse YAML frontmatter.
2. **Check** if the article already exists (by `article_id` or title match).
3. **Fork** a new version if the current version is already published.
4. **Update** the article content via the API.
5. **Publish** the new version with an auto-generated publish message.
6. **Apply flags** (e.g., "New" or "Updated") with configurable expiry.

Content sanitization

Every sync operation automatically:

- Strips Azure SAS tokens from CDN image URLs
- Removes `height` and `width` attributes from HTML image tags
- Normalizes `target="_blank"` attribute formatting
- Cleans trailing whitespace and normalizes line endings

Supporting scripts

Beyond the core CLI tool, I specified and directed the creation of ~60 additional Python scripts for:

Category	Examples
Content migration	HTML-to-Markdown conversion, bulk frontmatter injection
Schema analysis	XSD diff parsing for release note authoring
Quality checks	Link validation, image path verification, duplicate detection
Bulk operations	Mass category creation, article ordering, overview downloads
Reporting	Structure comparison, missing article detection

Portal customization

I also maintain custom CSS and JavaScript for the documentation portal, controlling:

- Navigation styling and layout
- Code block presentation
- Admonition/callout formatting
- Mobile responsiveness adjustments

Results

- **320+ articles** managed through the pipeline
- **6 API products** documented from a single repository
- **Automated deployment** — merge to main triggers publish within minutes
- **Zero manual CMS editing** — all content changes flow through Git
- **5+ years** of release notes published through this workflow